

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САУ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Техническое зрение»
Тема: Морфологические преобразования

Студент гр. 2491

Недовизий А.И.

Преподаватель

Федоркова А.О.

Санкт-Петербург

2025

Цель работы: научиться применять морфологические преобразования для исправления изображений и видео

Задания к работе:

Задание 1.

Напишите свою реализацию операций замыкания и размыкания для нескольких итераций. Каждую операцию оформите в виде отдельной функции (допустимый вариант - одна функция для двух операций, где тип операции задается через аргумент). Функция должна принимать исходное изображение, ядро и другие аргументы при необходимости. В вашей реализации замыкания и размыкания можно (нужно) использовать функции эрозии и наращивания, реализованные в OpenCV. На данном изображении добейтесь полной заливки кругов с использованием первой функции. С помощью операции размыкания удалите всё содержимое кругов на исходном изображении, затем сохраните его на компьютер.

Задание 2.

Для выполнения этого задания используйте изображение с шумом: точки на фоне или разрывы на границах.

Примените морфологические преобразования к такому изображению так, чтобы максимально убрать шум в виде точек на фоне и при этом закрыть разрывы в линиях текста.

Ход выполнения работы:

Задание 1:

Будет реализована одна функция с аргументом позволяющим выбрать тип операции. Код программы приведен в листинге 1.

Листинг 1

```
import numpy as np
import cv2

def erode_and_dilate(
    image, mode, iterations, kernel=np.ones((3, 3), np.uint8), anchor=None
) -> np.ndarray:
    image_new = image
    if mode == "open": ## размыкание
        for _ in range(iterations):
            image_new = cv2.erode(image_new, kernel, anchor)
        for _ in range(iterations):
            image_new = cv2.dilate(image_new, kernel, anchor)
    elif mode == "close": ## замыкание
        for _ in range(iterations):
            image_new = cv2.dilate(image_new, kernel, anchor)
        for _ in range(iterations):
            image_new = cv2.erode(image_new, kernel, anchor)
    else:
        print("wrong mode")

    return image_new

def show_img_and_wait(image, win_name, lifetime=0):
    cv2.imshow(win_name, image)
    cv2.waitKey(lifetime)

def main():
    path1 = r"/Users/new/Desktop/course4labs/cv/lab2/2-1.jpg"
    image = cv2.imread(path1, cv2.IMREAD_GRAYSCALE)
    cv2.imshow("sourcePic", image)
    cv2.waitKey(0)
    # fill given image
```

```

fill_image = erode_and_dilate(image=image, mode="open", iterations=2)
show_img_and_wait(fill_image, "fill circles custom")
pr_fill_image = cv2.morphologyEx(
    image, op=cv2.MORPH_OPEN, kernel=np.ones((3, 3), dtype=np.uint8), iterations=2
)
show_img_and_wait(pr_fill_image, "fill circles lib")

##delete everything inside the circle
clear_img = erode_and_dilate(
    image=image, mode="close", iterations=2, kernel=np.ones((3, 3), dtype=np.uint8)
)
show_img_and_wait(clear_img, "del from circles custom")

pr_clear_img = cv2.morphologyEx(
    image, op=cv2.MORPH_CLOSE, kernel=np.ones((3, 3), dtype=np.uint8), iterations=2
)
show_img_and_wait(clear_img, "del from circles lib")

##saving image
try:
    cv2.imwrite("saved_image_with_cleared_circles.png", clear_img)
    print("saved successfully")
except:
    print("smth went wrong")

if __name__ == "__main__":
    main()

```

На рисунке 1 представлены: изначальное изображение, результат работы собственной функции в режиме размыкание, результат работы функции в режиме замыкание соответственно.

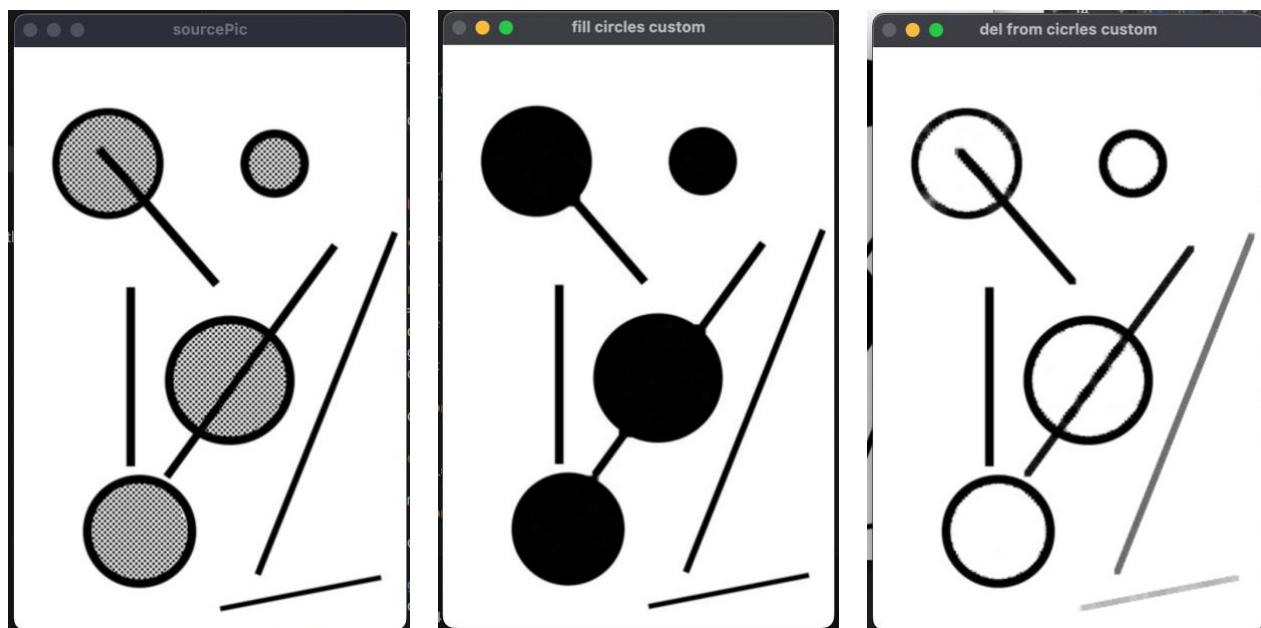


Рисунок 1 – Изображения полученные в процессе работы программы: исходное изображение, работа функции в режиме размывание, работа функции в режиме замыкания.

Как видно из полученных изображений, с помощью реализованных в собственной функции режимов замыкания и размывания, основанных на комбинациях наращивания и эрозии, нам удалось как полностью очистить содержимое кругов, так и наоборот их заполнить.

Задание 2: Код программы приведен в листинге 2.

Листинг 2:

```
import numpy as np
import cv2

def erode_and_dilate(
    image, mode, iterations, kernel=np.ones((3, 3), np.uint8), anchor=None
) -> np.ndarray:
    image_new = image
    if mode == "open": ## размывание
        for _ in range(iterations):
            image_new = cv2.erode(image_new, kernel, anchor)
```

```

    for _ in range(iterations):
        image_new = cv2.dilate(image_new, kernel, anchor)
    elif mode == "close": ##зamykanie
        for _ in range(iterations):
            image_new = cv2.dilate(image_new, kernel, anchor)
        for _ in range(iterations):
            image_new = cv2.erode(image_new, kernel, anchor)
    else:
        print("wrong mode")

    return image_new

def show_img_and_wait(image, win_name, lifetime=0):
    cv2.imshow(win_name, image)
    cv2.waitKey(lifetime)

def main():
    path2 = r"/Users/new/Desktop/course4labs/cv/lab2/2-2.jpg"
    image = cv2.imread(path2, cv2.IMREAD_GRAYSCALE)
    cv2.imshow("sourcePic", image)
    cv2.waitKey(0)
    test_kernel = np.full(shape=(2, 2), fill_value=2)
    first_clear = erode_and_dilate(
        image=image, mode="close", iterations=1, kernel=test_kernel ## zamikanie
    )
    second_clear = cv2.dilate(first_clear, kernel=test_kernel, iterations=1) ## narashivanie
    show_img_and_wait(image=second_clear, win_name="decrease noise")

if __name__ == "__main__":
    main()

```

На рисунке 2 представлена изначальная картинка с шумами, линиями на разрыве страниц.

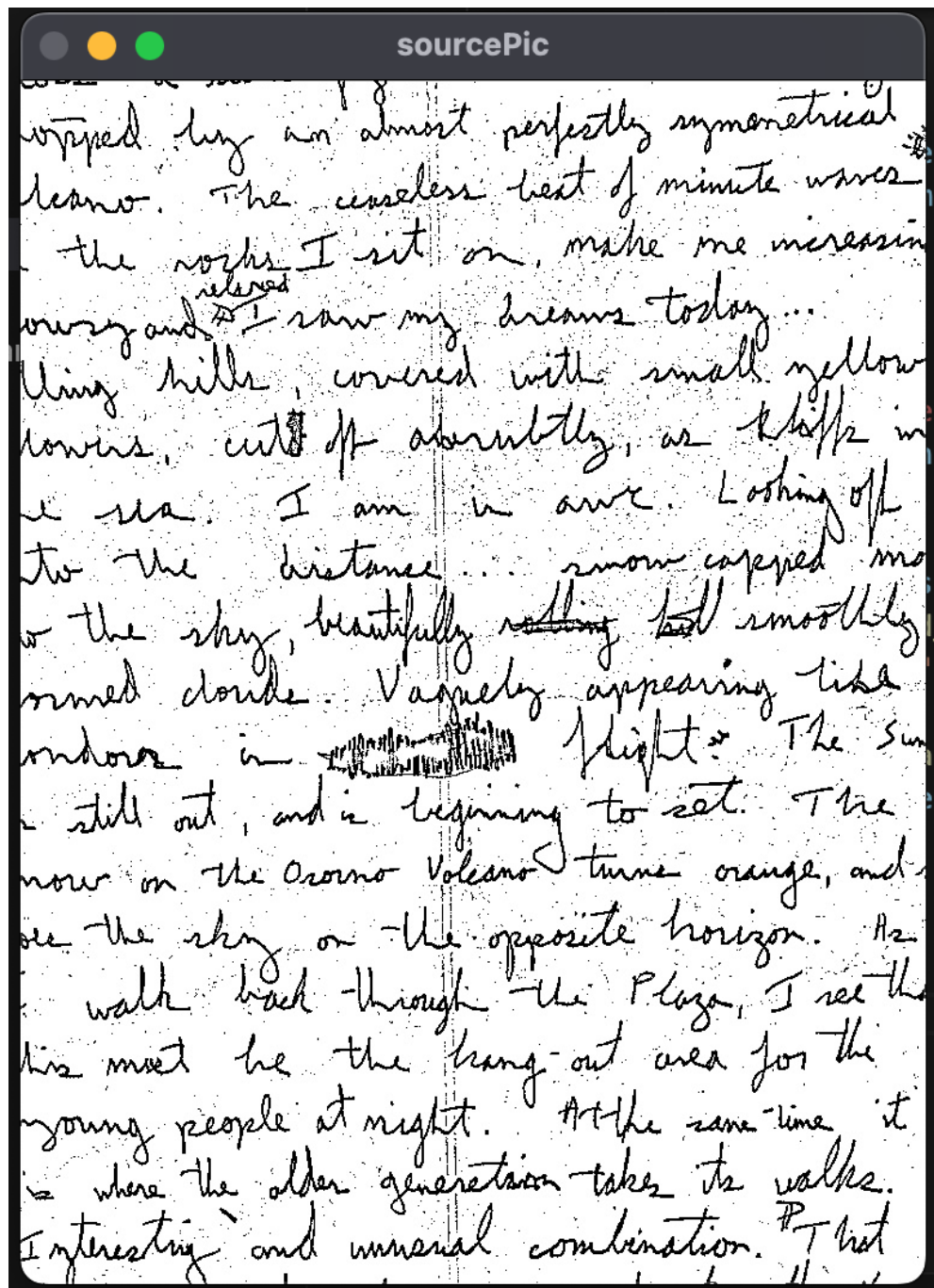


Рисунок 2 – Изначальная картинка с точками на фоне и разрывами страницы

На рисунке 3 представлена обработанная с помощью морфологических преобразований картинка, на которой гораздо меньше шумов. Для обработки было применено замыкание с 1 итерацией, а после него наращивание.

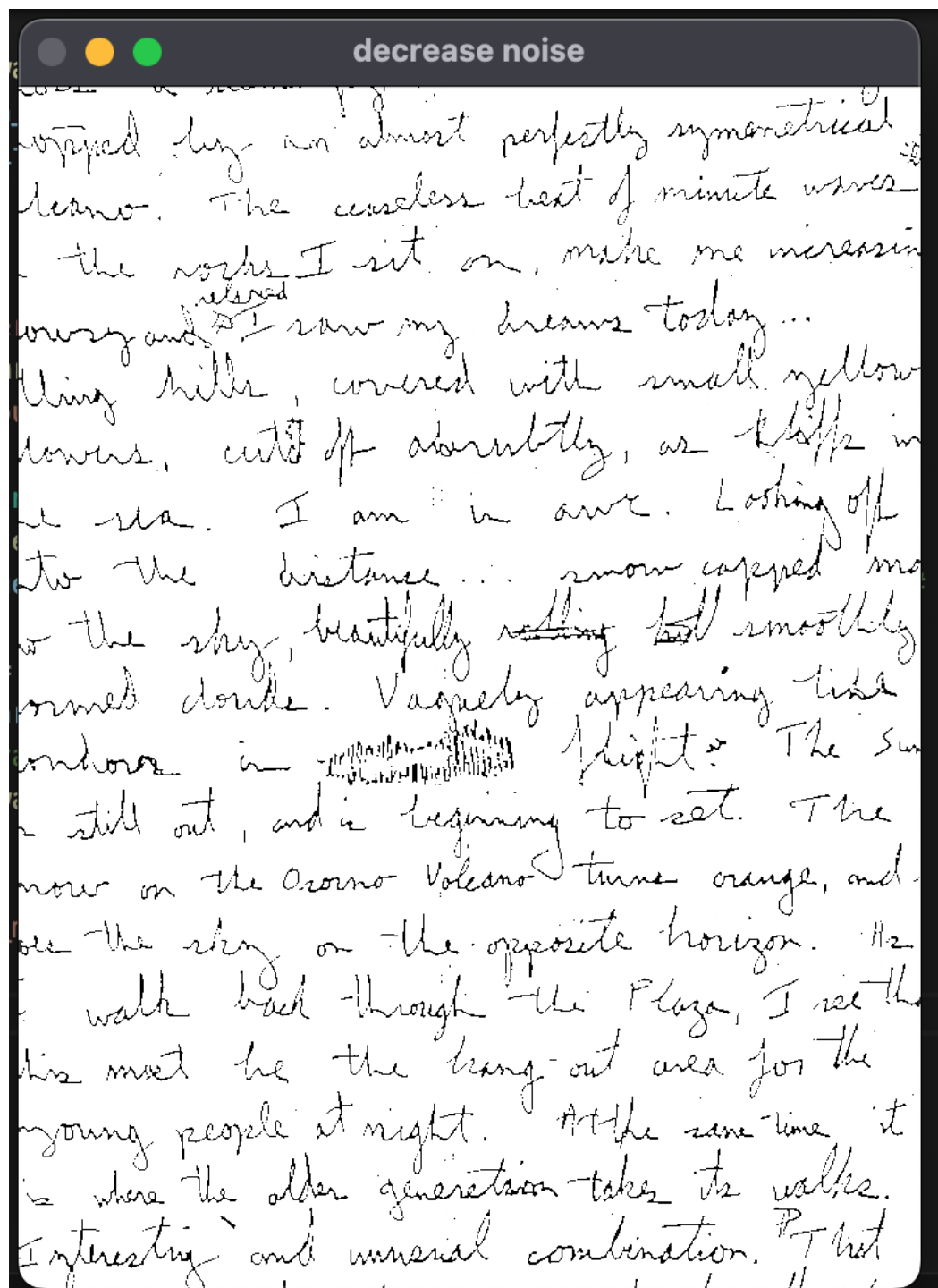


Рисунок 3 – Обработанная картинка, с уменьшением количества шума, и
очищением от разрыва страниц.

Вывод: в результате лабораторной работы были освоены основные
морфологические преобразования, такие как наращивание и эрозия. Также были

получены практические навыки работы с различными комбинациями таких этих морфологических преобразований такие как: размыкание и замыкание.